



香港中文大學計算機科學與工程學系

Department of Computer Science and Engineering

The Chinese University of Hong Kong

CSCI3360 Introduction to Computational Complexity

Course Logistics and Introduction

Instructor: [Xiao Liang](#)

Department of Computer Science and Engineering

The Chinese University of Hong Kong

(Nothing from this lecture will be tested on quizzes or exams!)

Plan for Week 1

Goal: Help you decide whether this elective course is right for you.

1. Review course logistics:
 - assessment methods,
 - the grading scheme,
 - other important information
2. Introduce computational complexity:
 - what it is,
 - why it is important,
 - the main topics covered in this course.
3. If time permits, begin the core content of Lecture 1.



Course Logistics

Course Logistics

All you need to know about this course:

1. The CUHK Blackboard site: course announcements and grades for quizzes, homework, project, and exams.
2. Course webpage: <https://xiao-liang.github.io/Courses/CSCI3360-26Fall/CSCI3360-26Fall.html>

A quick way to recover the link:

- Search for “Xiao Liang CUHK,” open the instructor’s homepage, and select the “Teaching” tab.

Important Information

What you can find on [the course page](#):

- TA information: office hour, office room, email, and related details.
- Time and venue information for lectures and tutorials.
- Weekly schedule and materials for lectures and tutorials.
- **Grading scheme:** Assessment information is summarized on the next slide.
- **Other mandatory information:** Make-up policy, regrading policy, academic integrity, SEN support, and use of AI tools.

➤ Grading Scheme (1/3)

Assessment consists of four parts:

1. Two Quizzes (10% each)
2. Midterm Exam (46%)
3. Group Homework (14%)
4. Group Project (20%)

Optional Exercise Sets:

- will be released as the course goes
- intended to give you enough practice, and help you get a sense of the types of questions that may appear on quizzes and exams
- You do not need to submit your solutions; they will not count toward your final grade.
- Some quiz or exam problems may be very similar to the exercises

➤➤ Grading Scheme (2/3)

Two Quizzes (10% each):

- will take place during lecture/tutorial hours.
- Each quiz is announced at least one week in advance.
- Closed-book
- But you can bring one A4-paper cheat sheet, double-sided, [hand-written](#)

Midterm Exam (46%):

- Held in the later weeks, for example between Weeks 11 and 13.
- Closed-book
- But you can bring one A4-paper cheat sheet, double-sided, [hand-written](#)

This course has no (centralized) final exam!

➤ Grading Scheme (3/3)

Group Homework (14%):

- **AI tools are not allowed**
- Two problem sets, each worth 7%. Students are allowed to form groups and submit one joint solution. More details will be announced later.
- Solutions must be prepared in LaTeX. [Overleaf](#) and its [Learn LaTeX in 30 minutes](#) guide are convenient starting points.

Group Project (20%):

- Students will work in groups to choose a complexity-theory paper published at a reputable conference (e.g., STOC/FOCS/SODA/ICALP), write a report, and give a presentation.

➤➤ Classes During Adverse Weather

See the course page for more detail.

In short:

- There will be no classes when the situation meets the university's conditions for the **Suspension of Classes**

A possible alternative is to hold the lecture online via Zoom.

By default, this course will **not** adopt this practice.

If you prefer to have online classes during adverse weather, please send me an email:

- If more than half of the enrolled students express this preference by email, we will switch to this online arrangement during adverse weather.
- I promise that I will not reveal the identity of anyone who sends such an email.

Any questions about the course logistics?



Introduction

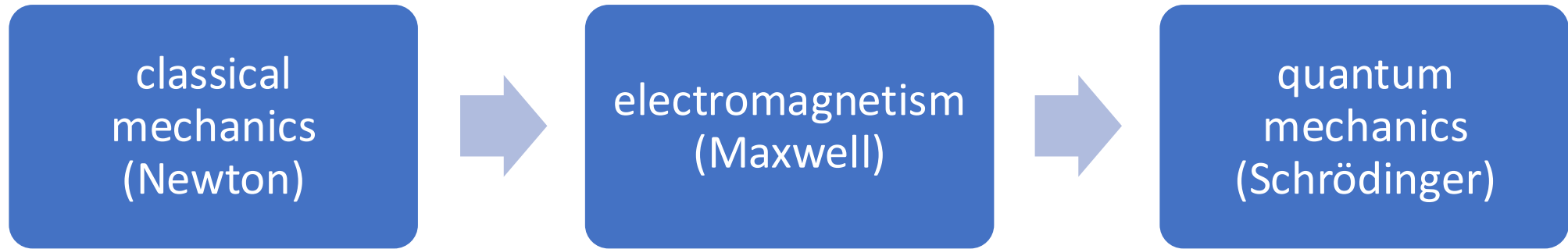


Part 1: Revolution of Computing

— From Newton to Schrödinger

➤➤ Core Idea: Computing Follows Physics

- Our computational power is bounded—and enabled—by our understanding of physics.
- Different physical laws suggest different ways to represent and process information.
- We will travel as



➤ Era I: Classical Mechanics and Mechanical Computing

Governing law for classical mechanics: Newton's second law

Force vector—the net force acting on an object.

$$\mathbf{F} = m \cdot \mathbf{a} = \frac{d\mathbf{p}}{dt}$$

Mass—a scalar measure of inertia

Acceleration vector

Momentum vector

time

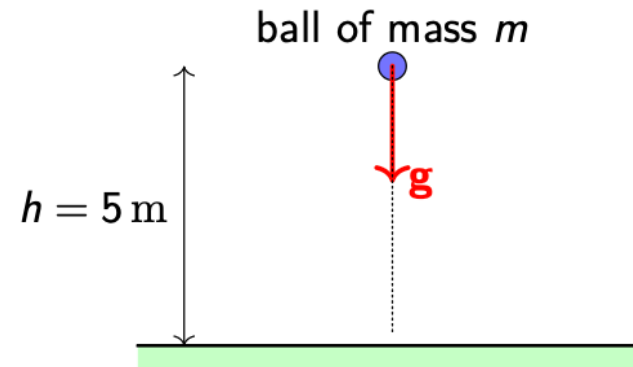
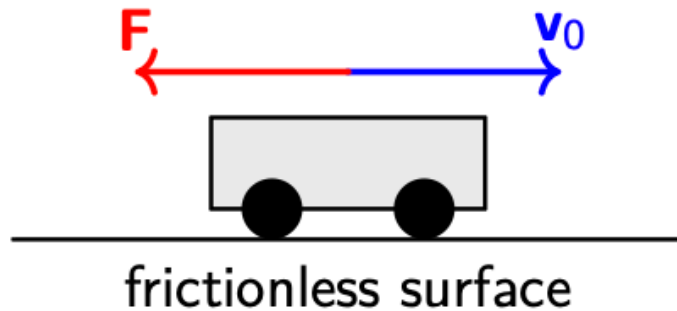
The time derivative of momentum is equal to the net force.

Newton's Second Law: Exemplary Systems

Governing law for classical mechanics: Newton's second law

$$\mathbf{F} = m \cdot \mathbf{a} = \frac{d\mathbf{p}}{dt}$$

Exemplary systems we can solve:

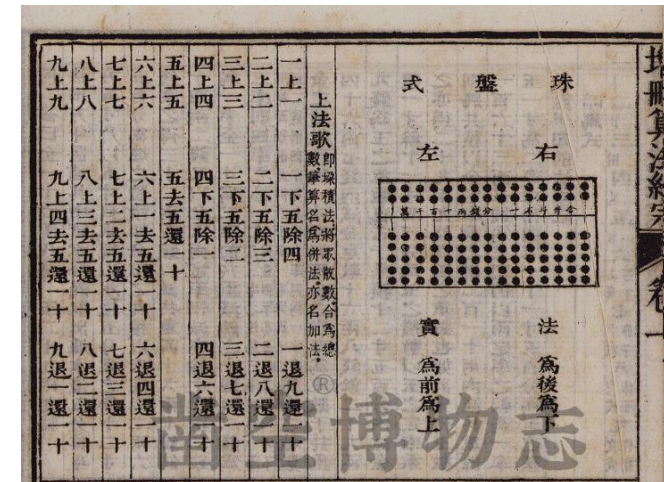
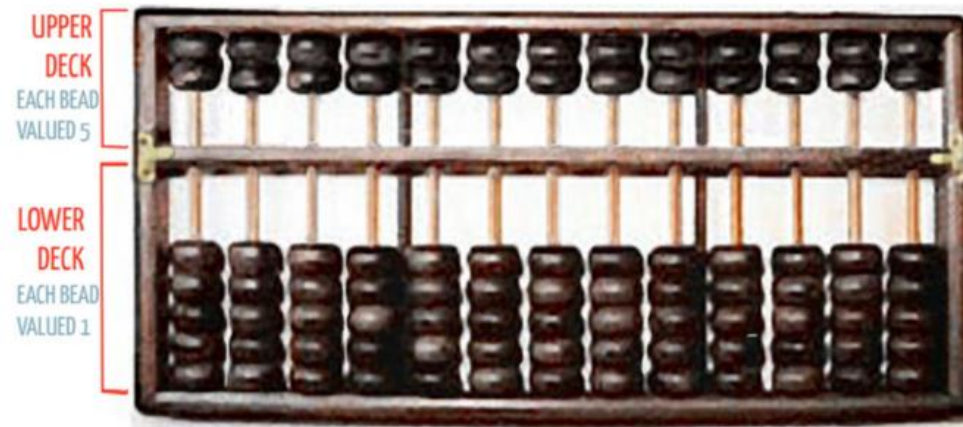


If we can build gears, levers, and pulleys, we can compute with them.

➤ Mechanical Computing Devices: Abacus

Abacus:

- Rods with beads represent digits.
- Computation is performed by moving beads according to rules.



Abacus and a Rulebook

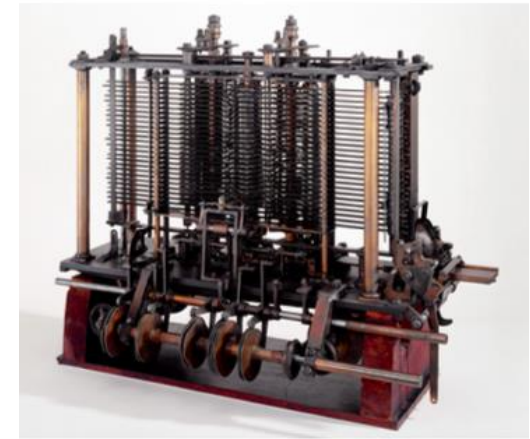
➤ Mechanical Computing Devices: Gear Calculators

Examples include the Pascaline and Babbage's analytical engine.

- Gear positions encode numbers.
- Addition and subtraction are performed through gear rotations and carries.



Pascaline



Babbage's analytical engine

➤ Era II: Electromagnetism and Electronic Computing

Governing laws: Maxwell's equations, unifying electricity and magnetism.

$$\nabla \cdot \mathbf{E} = \frac{\rho}{\varepsilon_0} \quad (\text{Gauss's law})$$

$$\nabla \cdot \mathbf{B} = 0 \quad (\text{Gauss's law for magnetism})$$

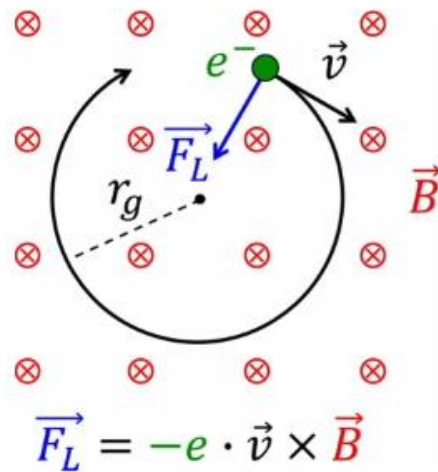
$$\nabla \times \mathbf{E} = - \frac{\partial \mathbf{B}}{\partial t} \quad (\text{Faraday's law})$$

$$\nabla \times \mathbf{B} = \mu_0 \mathbf{J} + \mu_0 \varepsilon_0 \frac{\partial \mathbf{E}}{\partial t} \quad (\text{Ampère–Maxwell law})$$

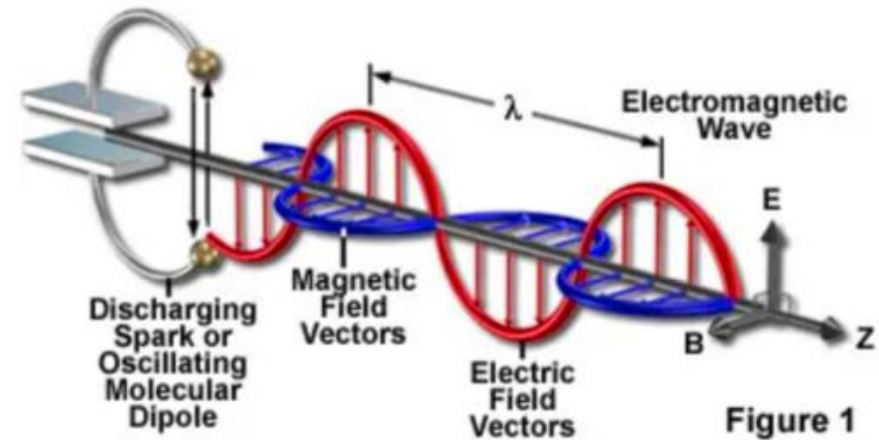
- $\mathbf{E}$: electric field
- $\mathbf{B}$: magnetic field
- ρ : charge density
- $\mathbf{J}$: current density
- ε_0 : the vacuum permittivity
- μ_0 : vacuum permeability.

➤ Systems Described by Maxwell's Equations

Exemplary systems we can solve using Maxwell's equations:



(a) An electron in a magnetic field



(b) Propagation of an electromagnetic wave

If we can harness fields and waves (wires, antennas, circuits), we can compute with electrons and light.

➤ Electronic Computing Devices

State-of-the-art devices include calculators, desktops, smartphones, and cloud servers.



➤ Era III: Quantum Mechanics and Quantum Computing

A New Revolution: **Quantum Mechanics**

- At microscopic scales, classical laws fail to explain experiments.
- Quantum mechanics provides new rules for system evolution and measurement.

Concrete examples:

- Atoms and molecules (about 0.1–1 nanometer): Explain elemental colors and chemical bonding.
- Tunneling (a few nanometers): Used in flash memory and scanning tunneling microscopes.
- Quantum dots (1–10 nanometers): Tiny crystals that produce different colors in modern displays.

➤➤ Schrödinger's Equation

Quantum mechanics is ruled by Schrödinger's equation.

$$i\hbar \frac{\partial}{\partial t} |\psi(t)\rangle = \hat{H} |\psi(t)\rangle$$

- It is the analogue of “ $F = m \cdot a$ ” in the quantum world.
- Further understanding requires a Quantum Mechanics course

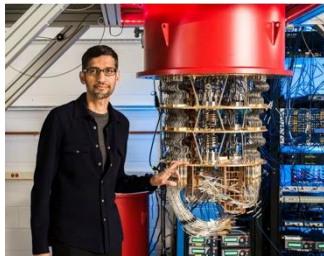
We can now build a new type of computers: Quantum Computers

Quantum is Coming – Computing Device (1/2)



2016

IBM -- Q5 (5 qubits)
First commercial quantum computer
Demo of concept



2019

Google -- Sycamore (53 qubits)
Experiment: Random Quantum Circuits Sampling
Advantage: 200 seconds vs 2.5 days



2020

USTC -- Jiuzhang (九章) (76 qubits)
Experiment: Boson Sampling
Advantage: 200 seconds vs 2.5 billion years



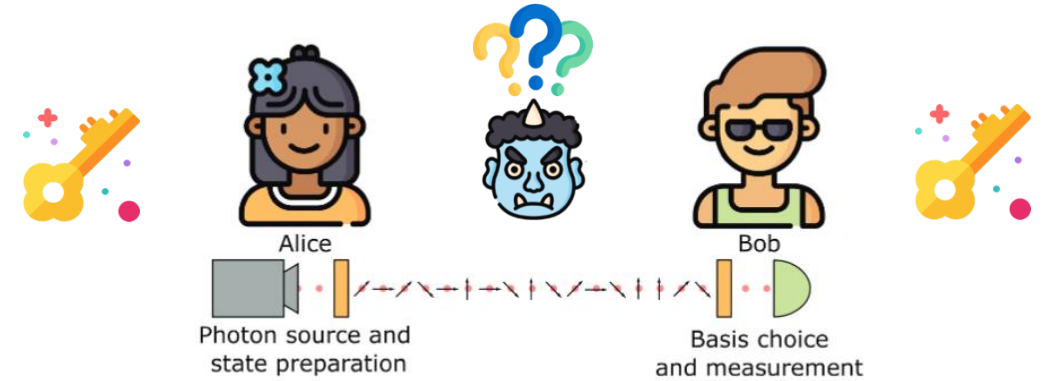
4 Dec, 2023

IBM -- Condor
1121 qubits!

Quantum is Coming – Computing Device (2/2)

Quantum Key Distribution (QKD)

First **flag-ship** application of quantum computing (Bennet-Brassard, 1984)



Commercially Available TODAY!

Price: ~ 82,000 US Dollars



ID Quantique – Cerberis

- up to 90 km
- 14,000 AES256 Keys per hour @ 18 dB loss



Toshiba – Long Distance QKD

- 150km
- 300 kb/s @ 10dB loss



QuantumCTek – PHA300

- 100km
- 50kbps @ 10dB loss

Advertisement!



Take the course:

[CSCI3350 Introduction to Quantum Computing](#)

➤➤ Era of AI: Large-Language Models



- No underlying new physics
- A revolution of computational power?
 - No consensus yet
- But LLMs have greatly changed the way we perceive computation

➤➤ A Motivating Question



Each revolution changes:

- How information is represented.
- Which elementary operations are available.
- How quickly some computations can be performed.

But what is the truth of computation?

Two Weird Examples (1/2)

The problem of Integer Factorization:

$$15 = 3 \times 5$$

$$211 = 13 \times 17$$

$$637057 = ? \quad (677 \times 941)$$

Classical computers

- No classical algorithm can solve integer factorization **efficiently**.
- It seems brute-forcing the prime number table is already the best.

Quantum computers

- Shor's algorithm solves it **efficiently** in the standard quantum model.

Changing the computational model can change whether a problem appears feasible.

Two Weird Examples (2/2)

Database Search Problem:

- Given an array of length n
- exactly one entry contains “1”
- all other entries contain “0”

Task: Write a program to find the address of the unique “1”



- A classical algorithm need to inspect all n positions.
- Grover's quantum algorithm only needs to inspect (roughly) $\sqrt{n}$ positions.

What is strange: it can be shown that $\sqrt{n}$ is the optimal for quantum computers!

➤➤ Back to the Motivating Question

Despite remarkable advances in computing technology, our fundamental understanding of computation is still limited!

What is the truth of computation?

- Does computation have to depend on physics?
- Can hard problems always be solved in future revolution of computing devices?
- Which computational limitations come from hardware? Which limitations come from the problem itself?
- What does it mean for something to be computable?
- Is there a difference between a mathematical solution and a solution implemented in real life using real-world devices?

Computational Complexity is the study of these “meta” problems!



Part 2: What This Course Studies

Four Ingredients of Complexity Theory

1

Models

- What is a computation?

2

Resources

- What is a computation?

3

Relationships

- How can one problem be transformed into another?

4

Limitations

- How can one problem be transformed into another?

We seek principles that remain meaningful when programming languages, hardware, and physics change.

Wait! Is this really possible?

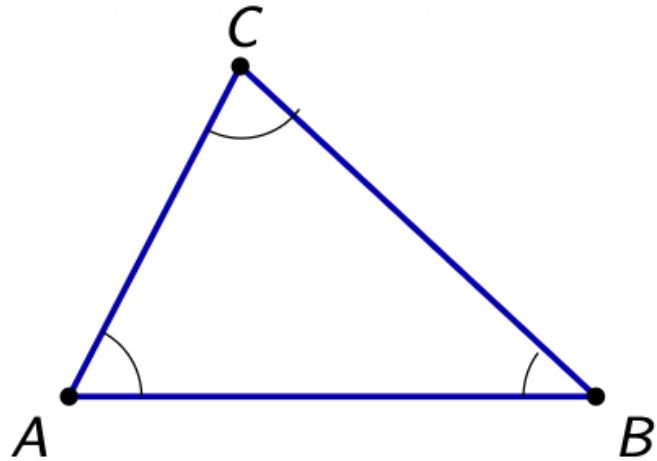
- Yes! Let me show you a successful example -- Gödel's Theorem



A Historical “Meta”-Problem: Gödel's Theorem

➤➤ A Familiar Mathematical System

Before discussing Gödel's theorem, recall Euclidean geometry.



In school, we learn statements such as:

- The angles of a triangle sum to 180 degree

But why are we allowed to conclude that this statement is true?

- A mathematical system: axiom, logic, proofs, and theorems

Axioms: Where Reasoning Begins

A mathematical system begins with statements accepted as starting assumptions. These statements are called **axioms**.

Axioms are the starting rules of the mathematical world we have chosen to study.

Euclidean geometry contains the following 5 axioms:

1. Exactly one straight line passes through two distinct points;
2. Any terminated straight line may be extended indefinitely.
3. A circle may be drawn with any given point as center and any given radius.
4. All right angles are equal.
5. Through a point outside a line, exactly one parallel line can be drawn.

➤ Logic: How We Move from One Statement to Another

Axioms alone are not enough. **Logic** specifies which reasoning steps are valid.

A Valid Example:

- Every square is a rectangle.
- This figure is a square.
- Therefore, this figure is a rectangle.

An Invalid Example:

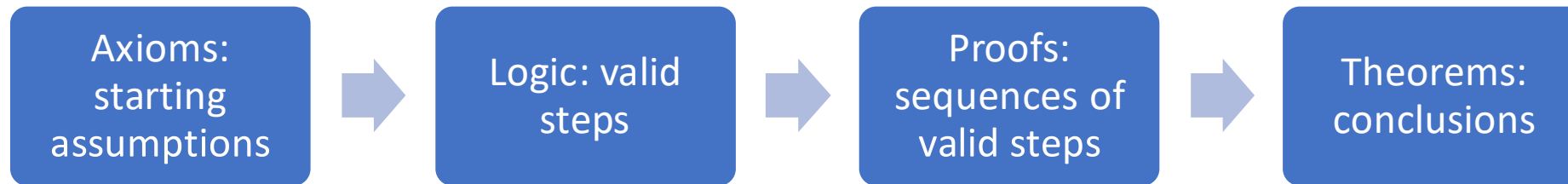
- Every square is a rectangle.
- This figure is a rectangle.
- Therefore, this figure is a square. (The wrong step)

Logic applies independently of whether the topic is geometry, arithmetic, or computation.

➤ Theorems and Proofs

- **Theorem:** A statement obtained from axioms through valid logical reasoning.
- **Proof:** The sequence of reasoning steps connecting axioms to the theorem.

In summary:

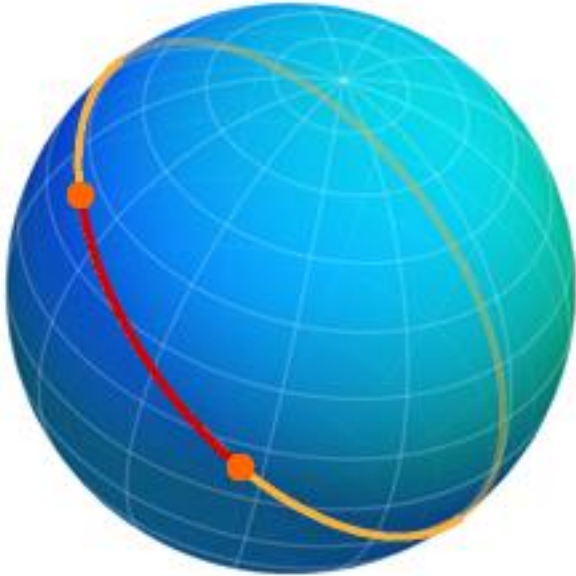


These four components constitute a mathematical system

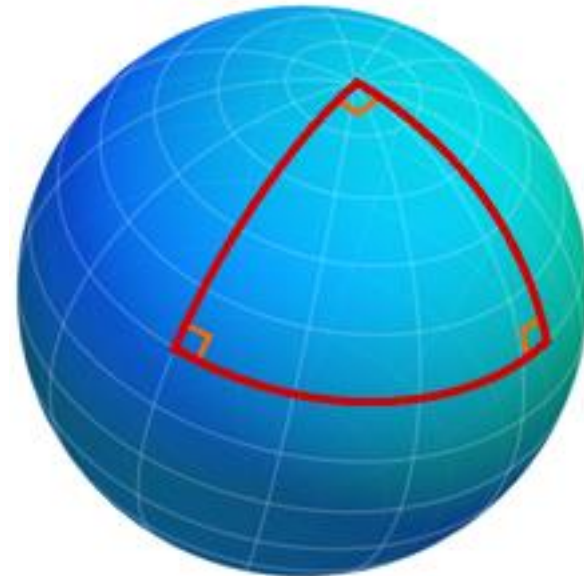
➤➤ Changing an axiom changes the mathematical world

Let's try to go against the 1st Axiom of Euclidean geometry:

1. Exactly one straight line passes through two distinct points;



The angles of a triangle do not always sum to 180 degrees.



Riemannian geometry:
the mathematical foundation of Albert Einstein's General Relativity.

➤ Criteria for a “Good” System

Two desirable properties of a reasonable math/formal system:

Soundness: everything it proves is actually true.

- Suppose our axioms are intended to describe the ordinary Euclidean plane.
- If the system proves a geometric statement, then that statement should be correct in the intended Euclidean plane.
- For example, if our system proves that every triangle has angles sum to 180 degrees, then this conclusion should really hold for every Euclidean triangle.

Completeness: A system is complete if every statement in its intended subject can be proved or refuted. I.e., no ambiguity!

Imagine asking every precisely stated question about the intended geometric world.

- Do the angles of every triangle sum to 180 degree?
- Does every pair of distinct points determine one line?
- Can a triangle have two right angles?

We would like the system to settle every such question.

➤ Gödel's Surprising Results

Soundness: everything it proves is actually true.

Completeness: A system is complete if every statement in its intended subject can be proved or refuted.

Gödel's Incompleteness Theorem:

- Any sufficiently powerful mathematical system cannot be both **complete** and **sound**.
-
- We will not discuss the proof here.
 - The proof relies on self-reference, an important technique that we will revisit in future lectures.
 - To give some intuition, we will illustrate the idea using **Russell's Paradox**.

➤ Russell's Paradox and Self-Reference (1/4)

Gödel's Incompleteness Theorem:

- Any sufficiently powerful mathematical system cannot be both **complete** and **sound**.

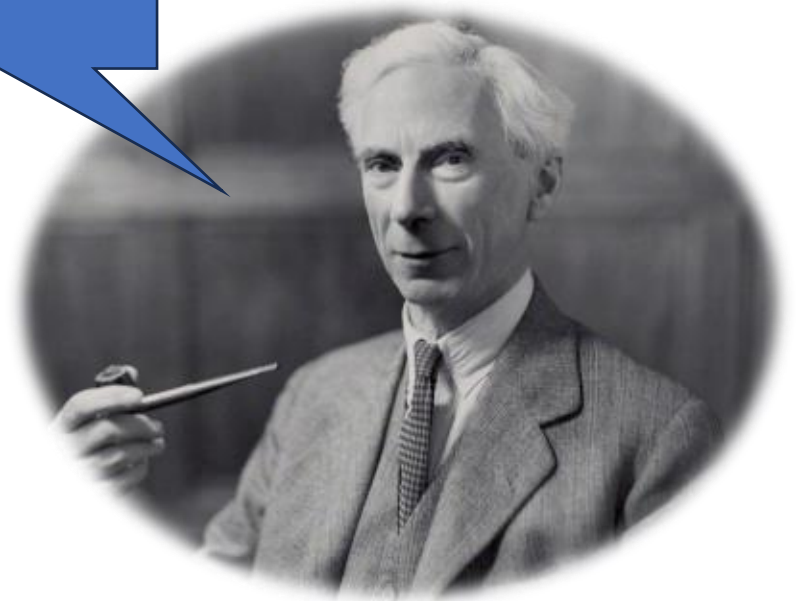
Wait! That is to say ...

- The system of Mathematics is problematic?!

Unfortunately, YES!

- Can you see the problem?

Let me tell you!



➤ Russell's Paradox and Self-Reference (2/4)

The most basic mathematical object is a **set**.

- The entire edifice of mathematics is built on this fundamental concept.
- By adding operations or structures to sets, we obtain more complex objects, such as groups, rings, fields, topologies, manifolds, functions, and more.

But what is a **set**?

A **set** is a collection of objects with some common property.

In early set theory, people used the following naive principle:

- For any property P , there is a set containing exactly the objects satisfying P .

For example:

The set of all natural numbers:

$$\{x : x \text{ is a natural number}\}$$

The set of all red objects:

$$\{x : x \text{ is red}\}$$

➤ Russell's Paradox and Self-Reference (3/4)

A **set** is a collection of objects with some common property.

- This seems natural, but it leads to a contradiction.

Consider the set:

$$R = \{x : x \text{ is a set and } x \notin x\}$$

In words, R is the set of all sets that **do not contain themselves**.

Now ask:

$$R \in R ?$$

- If $R \in R$, then by the definition of R , it must satisfy $R \notin R$.
- If $R \notin R$, then R satisfies the defining property of R , so $R \in R$.

Therefore:

$$R \in R \iff R \notin R$$

This is a contradiction.

➤ Russell's Paradox and Self-Reference (4/4)

Barber's Paradox: An Intuitive Analogy of Russell's Paradox

Imagine a town with one barber. There is a weird rule in this town:

- The barber shaves exactly those people who **do not shave themselves**.

Now ask:

- Does the barber shave himself?

Two cases:

1. If the barber shaves himself, then he is someone who shaves himself, so by the rule, he should **not** shave himself.
2. If the barber does not shave himself, then he is someone who does not shave himself, so by the rule, he **should** shave himself.

Again, we get a contradiction.

Such a technique to construct contradiction is called “self-reference”.

We will use this technique again later in the lecture.

➤ From Gödel and Russell back to Our Course

Gödel's theorem and Russell's paradox illustrate what meta-problems look like, and how we can study them.

Object-level question

- Can we prove this particular logical/mathematical statement?

Meta-level question

- Can this proof system settle every arithmetic statement?

Computational complexity studies analogous questions:

- Can any algorithm solve every instance quickly?
- How can we prove limitations that apply to all algorithms?
- and much more...



Part 3: Why Take This Course?

➤ Computer Science Is Not Only Programming

Programming is essential, but computer science is not simply the study of software tools.

Engineering

- Build reliable systems.
- Improve performance.
- Use current platforms.
- Solve practical problems.

Science

- Discover general principles.
- Build mathematical models.
- Prove what is possible.
- Prove limitations.

Complexity theory is one of the subjects that makes computation a science!

Why Offer This Course at CUHK?

Strong computer science programs provide substantial opportunities to study theoretical foundations.

Theory is essential for students interested in:

- Algorithms and optimization.
- Cryptography and security.
- Quantum computing.
- Machine learning theory.
- Research and graduate study.

Building a Broader Theoretical Curriculum

We have received feedback from students in the theory stream.
The department is making efforts to address these concerns.

New theoretical CS courses installed in the last 2 years:

- **CSCI3350:** Introduction to Quantum Computing
- **CSCI3360:** Introduction to Computational Complexity
- **CSCI4240:** Advanced Algorithms: Randomization and Approximation (co-taught with Prof. [Hanrui Zhang](#))
- **CSCI5350:** Advanced Topics in Game Theory (by Prof. [Hanrui Zhang](#))
- **CSCI5440:** Theory of Cryptography

More to come. Stay tuned.

A simple principle:

- These courses are offered in the curricula of leading universities such as MIT, Harvard, Berkeley, Oxford, and Cambridge. Our CUHK students should have the opportunity to take them as well!

➤➤ What you will gain from this course (1/2)

This course develops **mathematical tools** used throughout theoretical computer science:

- Precise definitions and mathematical modeling.
- Asymptotic analysis.
- Proof by contradiction and diagonalization (self-reference).
- Reductions between computational problems.
- Simulation between models.
- Counting and probabilistic arguments.
- Adversarial reasoning.
- Upper and lower bounds.

➤➤ What you will gain from this course (2/2)

This course gives you **a complexity-oriented perspective**

Before immediately starting to code, ask:

1. What is the precise computational problem?
2. What is the input size?
3. Do we require an exact answer?
4. Would an approximation be sufficient?
5. Is the problem related to a known difficult problem?
6. Are practical instances specially structured?
7. Can a proposed answer be checked more easily than it can be found?

Formalizing the correct problem may be more important than optimizing the first program.

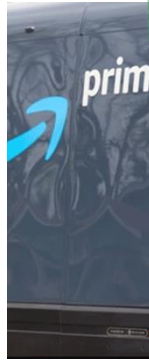
- Let's see 3 concrete examples.



Example 1: A Delivery Company's Request

➤ An Imaginary First Day at

Assume you join one of these companies after graduation:



Your boss is not happy with the current routing for delivery.

- Why?
- He asks you to optimize the delivery route

Modeling the Problem

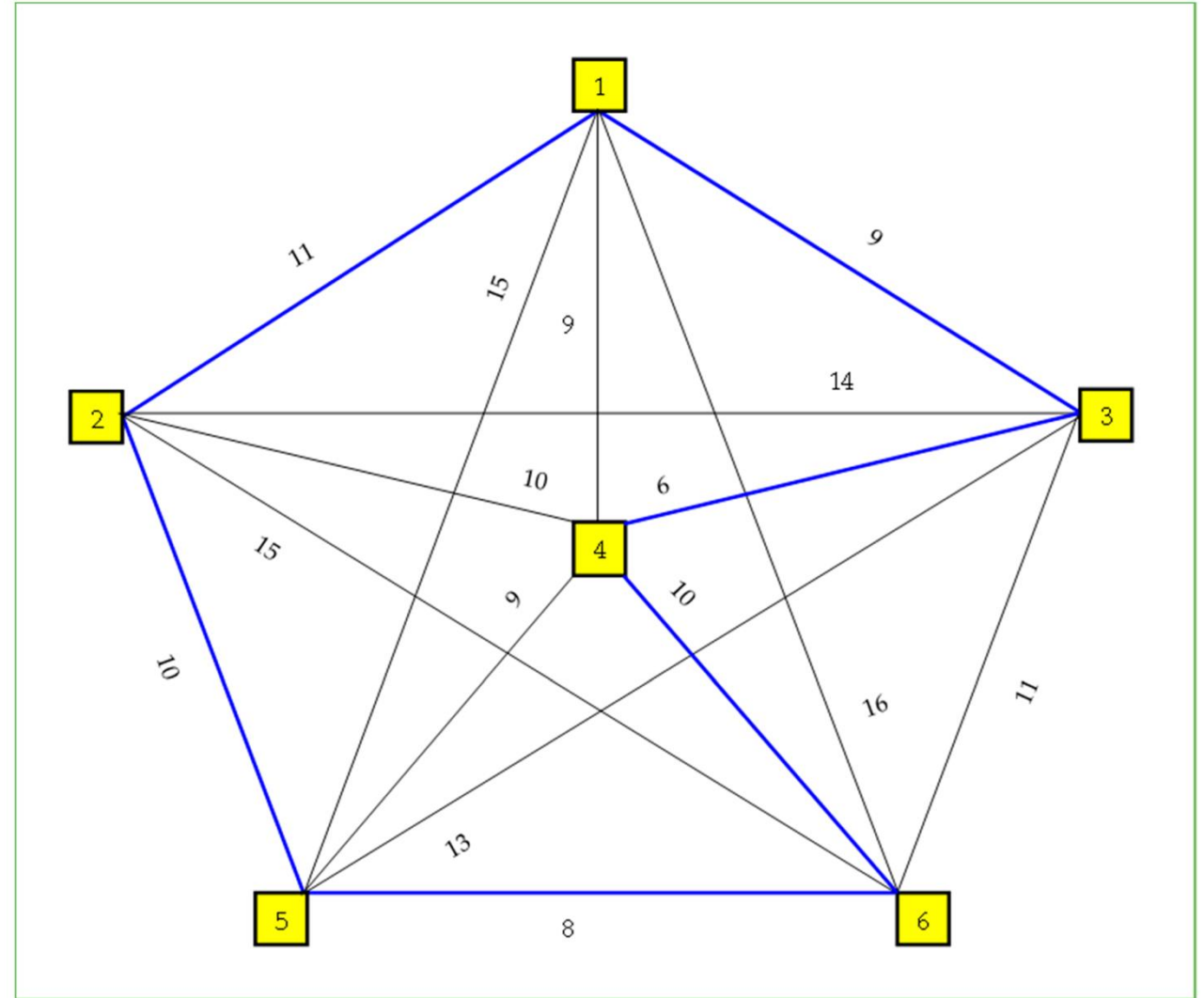
A graph consists of:

- Vertices: representing delivery locations;
- Edges: representing direct routes
- Weights: cost (e.g., gas fee or physical distance)

Your boss wants a route that

- visits every vertex exactly once.
- returns to the starting vertex.
- has minimum total travel cost.

This optimization problem is the Traveling Salesperson Problem, or TSP.



➤➤ Why Trying Every Route Is Expensive

With n locations, the number of possible visiting orders grows roughly like:

$$n!$$

For example:

$$5! = 120, \quad 10! = 3,628,800, \quad 20! \approx 2.43 \times 10^{18}.$$

Even if a computer checked one billion routes per second, checking all $20!$ routes would take **many years**.

A clever algorithm may avoid checking every route.

But can we design an algorithm that always finds the optimal route efficiently?

TSP is NP-Hard

The Traveling Salesman Problem is “NP-hard”

A problem is NP-hard if an efficient algorithm for it would also provide efficient algorithms for a large family of famous difficult problems.

- NP-hardness is stronger evidence than “I wrote a program, and it was slow.”
- We will formally study this notion during this semester

➤➤ What Should We Tell Your Boss?

NP-hardness does not mean useful software is impossible.

Instead, we should discuss alternatives with your boss

- Can we accept a route within a few percent of optimal?
- Can we solve small regions separately?
- Can a heuristic provide a good route quickly?
- Can an exact exponential-time algorithm handle the actual input size?

Complexity theory helps reformulate an unrealistic request into a meaningful computational goal.



Example 2: Lattin Square

➤ The Story Begins with an Online Assessment

Around 2017, a friend was preparing for an online recruitment assessment.

- PWC (羅兵咸永道 in Hong Kong, or 普华永道 in Mainland China)

A puzzle is called **Latin Square**:

- Given an $n \times n$ grid. Some entries are already fixed.

The task is to complete the empty cells so that:

- every symbol appears exactly once in each row;
- every symbol appears exactly once in each column.

The sample assessment contained **timed grid puzzles**.

- It began with small 3×3 puzzles.
- Solving several quickly led to 4×4 puzzles.
- Further success led to 5×5 puzzles.

The goal was to solve as many as possible within a short time.

	1	3
1		



3	2	1
2	1	3
1	3	2

	1		3
		4	
3			

The larger,
the harder

➤ First Attempt: Brute Force

My friend asked whether I could write a Python program for the puzzle.

My first program tried possible values in the empty cells.

If there are k empty cells and n possible symbols per cell, the naive search may examine:

$$n^k$$

assignments.

For example:

$$n = 4, \quad k = 12 \Rightarrow 4^{12}$$

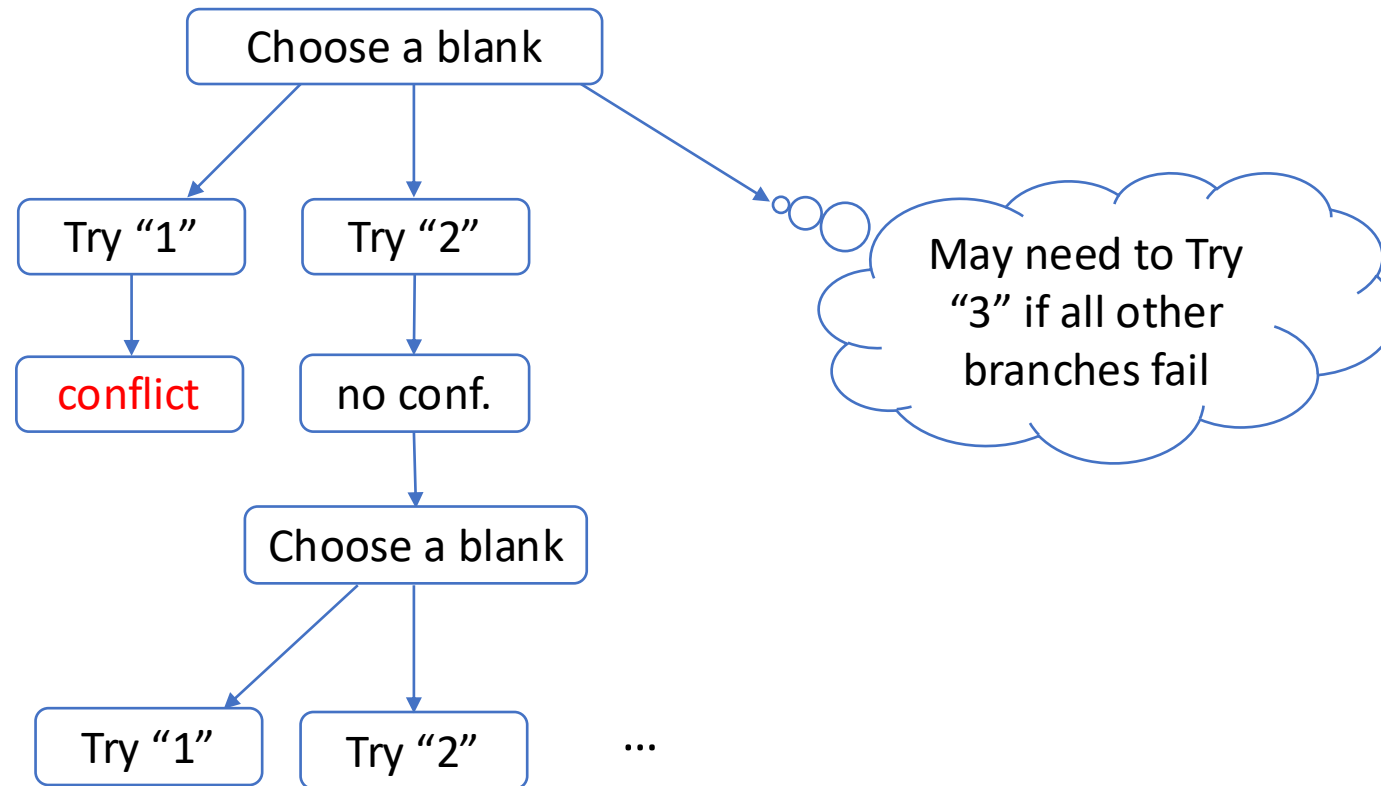
The program solved many small instances.

However, it could become stuck even on certain 3×3 instances.
Its performance depended strongly on the clues.

➤➤ Second Attempt: Backtracking

I then used backtracking:

- As soon as a choice violates a row or column constraint, the program abandons that branch.



Where I got stuck

Backtracking performed much better.

- It solved the 3×3 practice instances quickly.
- It handled many 4×4 and 5×5 instances.
- Compared with brute force, it felt as though the problem had been solved.

At 6×6 , the program became unreliable and extremely slow.

I tried several improvements:

- changing the order of the empty cells;
- adding more pruning rules;
- searching for better strategies.

I spent several days on the problem. Eventually, I gave up.

My conclusion was:

- “Perhaps I’m simply not clever enough to find the right algorithm.”

➤➤ What Complexity Theory Later Told Me

In 1984, Charles Colbourn proved that partial Latin square completion is **NP-hard**.

The story distinguishes three statements.

1. **My program is slow:** This may be only an implementation problem.
2. **Every program I tried is slow:** This is empirical evidence, but not a theorem.
3. **The problem is NP-complete:** This connects it mathematically to a large family of difficult problems.

Complexity theory moves us from personal programming experience to general statements about computational problems.

A Brief Ethical Note

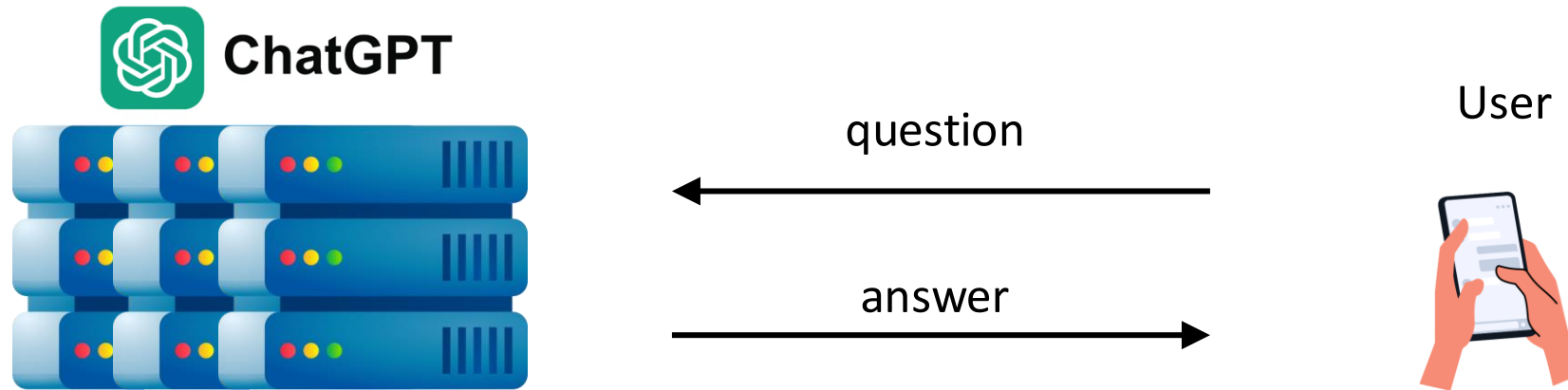
- Using unauthorized software during a real assessment may violate the rules.
- Candidates should always follow the stated assessment rules.



Example 3: Can We Verify a Paid LLM Service?

➤➤ A Familiar Real-World Situation

Typical way of Interaction between users and LLM service providers



The most capable LLMs may require:

- Enormous memory for parameters.
- Specialized hardware such as GPUs.
- Substantial computational power.
- Considerable inference time and energy.

➤ Different Subscription Levels

ChatGPT

Pricing

See pricing for our individual, business, and enterprise plans.

Individual Business & Enterprise

Free

Intelligence for everyday tasks

\$0 / month

[Get Free >](#)

- ✓ Unlimited text chats with GPT-5.6 Luna
- ✓ Limited messages with uploads
- ✓ Limited and slower image generation
- ✓ Limited voice chats

Go

Expanded access

\$8 / month

[Get Go >](#)

Everything in Free and:

- ✓ More messages with tools
- ✓ More uploads
- ✓ More image creation

Plus

Do more with advanced intelligence

\$20 / month

[Get Plus >](#)

Everything in Go and:

- ✓ Advanced reasoning models with GPT-6 Astra and GPT-5.6
- ✓ Expanded messages and uploads

Pro

Maximize your productivity

From **\$100** / month

[Get Pro >](#)

Everything in Plus and:

- ✓ 5x or 20x more usage
- ✓ Pro reasoning powered by GPT-6 Astra
- ✓ Maximum CoCox tasks

Models & pricing > Lifecycle and reference

Pricing

Learn about Anthropic's pricing structure for models and features

This page provides detailed pricing information for Anthropic's models and features. All prices are in USD.

For the most current pricing information, visit claude.com/pricing.

Model pricing

The following table shows pricing for all Claude models:

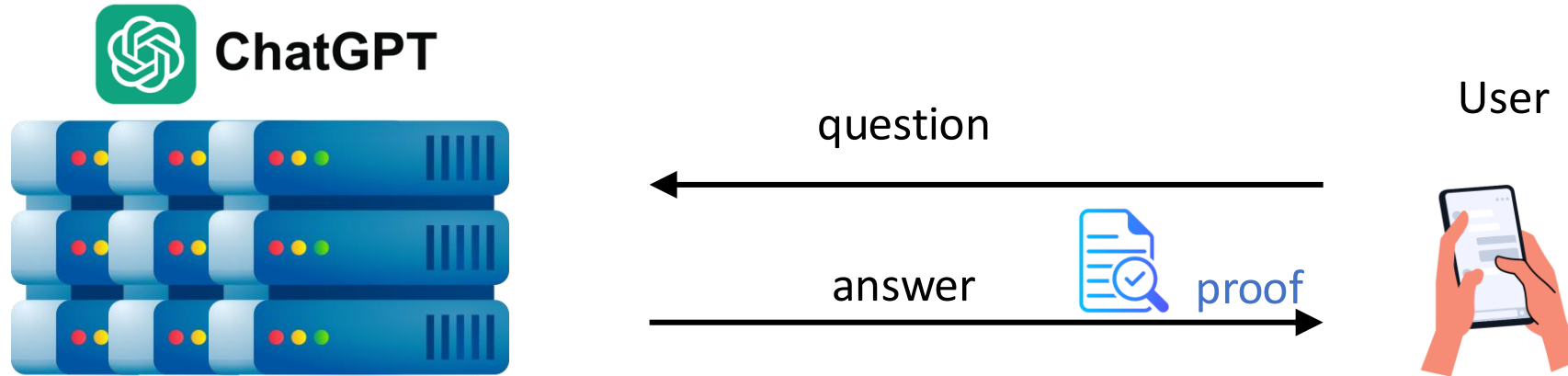
Model	Base input tokens	5m cache writes	1h cache writes	Cache hits and refreshes	Output tokens
Claude Fable 5.1	\$10 / MTok	\$12.50 / MTok	\$20 / MTok	\$0.25 / MTok ¹	\$50 / MTok
Claude Mythos 5.1 (limited availability)	\$10 / MTok	\$12.50 / MTok	\$20 / MTok	\$0.25 / MTok ¹	\$50 / MTok
Claude Fable 5	\$10 / MTok	\$12.50 / MTok	\$20 / MTok	\$1 / MTok	\$50 / MTok
Claude Mythos 5 (limited availability)	\$10 / MTok	\$12.50 / MTok	\$20 / MTok	\$1 / MTok	\$50 / MTok
Claude Opus 5	\$5 / MTok	\$6.25 / MTok	\$10 / MTok	\$0.50 / MTok	\$25 / MTok
Claude Opus 4.8	\$5 / MTok	\$6.25 / MTok	\$10 / MTok	\$0.50 / MTok	\$25 / MTok
Claude Opus 4.7	\$5 / MTok	\$6.25 / MTok	\$10 / MTok	\$0.50 / MTok	\$25 / MTok
Claude Opus 4.6	\$5 / MTok	\$6.25 / MTok	\$10 / MTok	\$0.50 / MTok	\$25 / MTok
Claude Opus 4.5	\$5 / MTok	\$6.25 / MTok	\$10 / MTok	\$0.50 / MTok	\$25 / MTok
Claude Opus 4.1 (retired, except on Bedrock and Google Cloud)	\$15 / MTok	\$18.75 / MTok	\$30 / MTok	\$1.50 / MTok	\$75 / MTok

A premium customer expects the company to use the promised premium model

But the customer sees only the prompt and returned answer

Can we verify if the LLM provider really serves you with the promised model?

Verifiable Delegated Computation



User can check the proof to obtain the following guarantees:

- Cheating service provider will always be caught
- The proof should be **easy to check** --- much cheaper than the asserted computation itself.
 - E.g., an $O(n)$ time computation can be verified in time, say, $O(\log n)$

An active research area initiated by:

- [Delegating computation: interactive proofs for muggles](#), S. Goldwasser, Y.T. Kalai, G.N. Rothblum.
Journal of the ACM (JACM) 62 (4), 1-64. 2015

Wait! Is this really possible?

Is the following really possible?

- an $O(n)$ time computation can be verified in time $O(\log n)$

That means, we don't need to read the full proof to be convinced of the correctness of a theorem???

- No, we don't!

PCP Theorem: Every efficiently checkable proof can be encoded in a manner such that a verifier can be convinced by reading only constantly many randomly-chosen places of it.

- PCP stands for “probabilistically checkable proofs”
- One of the most celebrated results in Computer Science

A proof of Cauchy-Schwarz Inequality

Proof

Suppose X and Y are two random variables and, assuming their variances are positive, then the correlation of (X, Y) is defined as

$$r = \frac{\text{Cov}(X, Y)}{\sqrt{\text{Var}(X) \cdot \text{Var}(Y)}}; (-1 \leq r \leq 1)$$

Squaring the coefficient of correlation gives coefficient of determination, which means

$$r^2 = \frac{\left(\frac{\sum (X - \bar{X})(Y - \bar{Y})}{n} \right)^2}{\frac{\sum (X - \bar{X})^2}{n} \cdot \frac{\sum (Y - \bar{Y})^2}{n}} \leq 1$$

$$\text{Or } \left[\sum (X - \bar{X})(Y - \bar{Y}) \right]^2 \leq \left[\sum (X - \bar{X})^2 \right] \cdot \left[\sum (Y - \bar{Y})^2 \right]$$

$$\Rightarrow \sum [XY - \bar{Y}X - \bar{X}Y + \bar{X}\bar{Y}]^2 \leq \left[\sum (X - \bar{X})^2 \right] \cdot \left[\sum (Y - \bar{Y})^2 \right]$$

$$\Rightarrow \left[\sum XY - \bar{Y} \sum X - \bar{X} \sum Y + n\bar{X}\bar{Y} \right]^2 \leq \left[\sum (X - \bar{X})^2 \right] \cdot \left[\sum (Y - \bar{Y})^2 \right]$$

$$\Rightarrow \left[\sum XY - n\bar{X}\bar{Y} - n\bar{X}\bar{Y} + n\bar{X}\bar{Y} \right]^2 \leq \left[\sum (X - \bar{X})^2 \right] \cdot \left[\sum (Y - \bar{Y})^2 \right]$$

$$\Rightarrow \left[\sum XY - n\bar{X}\bar{Y} \right]^2 \leq \left[\sum (X - \bar{X})^2 \right] \cdot \left[\sum (Y - \bar{Y})^2 \right]$$

$$\Rightarrow \left[\sum XY \right]^2 \leq \sum X^2 \sum Y^2 - n \left[\sum X^2 (\bar{Y})^2 + (\bar{X})^2 \sum Y^2 - 2\bar{X}\bar{Y} \sum XY \right]$$

$$\Rightarrow \left[\sum XY \right]^2 \leq \sum X^2 \sum Y^2 - n \left[\left(\sum X \right) \bar{Y} - \left(\sum Y \right) \bar{X} \right]^2$$

$$\text{Since } \left[\sum X\bar{Y} - \sum Y\bar{X} \right]^2 > 0$$

$$\therefore \left[\sum XY \right]^2 \neq \sum X^2 \sum Y^2$$

$$\text{Or } \left[\sum XY \right]^2 < \sum X^2 \sum Y^2$$

$$\text{Hence proved.}$$

PCP Theorem

PCP Theorem: Every efficiently checkable proof can be encoded in a manner such that a verifier can be convinced by reading only constantly many randomly-chosen places of it.

Proof

Suppose X and Y are two random variables and, assuming their variances are positive, then the correlation of (X, Y) is defined as

$$r = \frac{\text{Cov}(X, Y)}{\sqrt{\text{Var}(X) \text{Var}(Y)}}; (-1 \leq r \leq 1)$$

Squaring the coefficient of correlation gives coefficient of determination, which means

$$r^2 = \frac{\left(\frac{\sum (X - \bar{X})(Y - \bar{Y})}{n} \right)^2}{\frac{\sum (X - \bar{X})^2}{n} \frac{\sum (Y - \bar{Y})^2}{n}} \leq 1$$

Or $\left[\frac{\sum (X - \bar{X})(Y - \bar{Y})}{n} \right]^2 \leq \left[\frac{\sum (X - \bar{X})^2}{n} \right] \left[\frac{\sum (Y - \bar{Y})^2}{n} \right]$

$$\Rightarrow \left[\sum (XY - \bar{Y}X - \bar{X}Y + \bar{X}\bar{Y}) \right]^2 \leq \left[\sum (X - \bar{X})^2 \right] \left[\sum (Y - \bar{Y})^2 \right]$$

$$\Rightarrow \left[\sum XY - \bar{Y} \sum X - \bar{X} \sum Y + n\bar{X}\bar{Y} \right]^2 \leq \left[\sum (X - \bar{X})^2 \right] \left[\sum (Y - \bar{Y})^2 \right]$$

$$\Rightarrow \left[\sum XY - n\bar{X}\bar{Y} - \bar{Y} \sum X + n\bar{X}\bar{Y} \right]^2 \leq \left[\sum (X - \bar{X})^2 \right] \left[\sum (Y - \bar{Y})^2 \right]$$

$$\Rightarrow \left[\sum XY - n\bar{X}\bar{Y} \right]^2 \leq \left[\sum (X - \bar{X})^2 \right] \left[\sum (Y - \bar{Y})^2 \right]$$

$$\Rightarrow \left[\sum XY^2 + n(\bar{X})^2(\bar{Y})^2 - 2n\bar{X}\bar{Y} \sum XY \right] \leq \left[\sum X^2 + n(\bar{X})^2 - 2\bar{X} \sum X \right] \left[\sum Y^2 + n(\bar{Y})^2 - 2\bar{Y} \sum Y \right]$$

$$\Rightarrow \left[\sum XY^2 + n(\bar{X})^2(\bar{Y})^2 - 2n\bar{X}\bar{Y} \sum XY \right] \leq \left[\sum X^2 + n(\bar{X})^2 - 2n\bar{X} \right] \left[\sum Y^2 + n(\bar{Y})^2 - 2n\bar{Y} \right]$$

$$\Rightarrow \left[\sum XY^2 + n(\bar{X})^2(\bar{Y})^2 - 2n\bar{X}\bar{Y} \sum XY \right] \leq \left[\sum X^2 - n(\bar{X})^2 \right] \left[\sum Y^2 - n(\bar{Y})^2 \right]$$

$$\Rightarrow \left[\sum XY^2 + n(\bar{X})^2(\bar{Y})^2 - 2n\bar{X}\bar{Y} \sum XY \right] \leq \left[\sum X^2 \sum Y^2 - n \sum X^2 \bar{Y}^2 - n(\bar{X})^2 \sum Y^2 + n^2(\bar{X})^2 \bar{Y}^2 \right]$$

$$\Rightarrow \left[\sum XY^2 - 2n\bar{X}\bar{Y} \sum XY \right] \leq \left[\sum X^2 \sum Y^2 - n \sum X^2 \bar{Y}^2 - n(\bar{X})^2 \sum Y^2 \right]$$

$$\Rightarrow \left[\sum XY^2 \right] \leq \left[\sum X^2 \sum Y^2 - n \sum X^2 \bar{Y}^2 - n(\bar{X})^2 \sum Y^2 + 2n\bar{X}\bar{Y} \sum XY \right]$$

$$\Rightarrow \left[\sum XY^2 \right] \leq \sum X^2 \sum Y^2 - n \left[\sum X^2 \bar{Y}^2 + (\bar{X})^2 \sum Y^2 - 2\bar{X}\bar{Y} \sum XY \right]$$

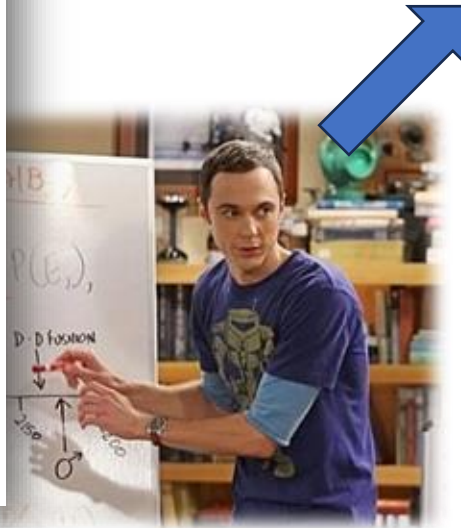
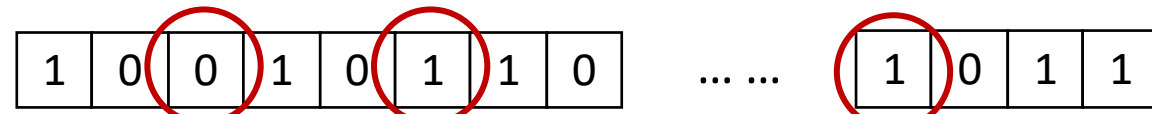
$$\Rightarrow \left[\sum XY^2 \right] \leq \sum X^2 \sum Y^2 - n \left[(\sum X)^2 \bar{Y}^2 - (\sum Y)^2 \bar{X}^2 \right]$$

Since $\left[\sum XY - \sum X \bar{Y} \right]^2 > 0$

$$\therefore \left[\sum XY \right]^2 \geq \sum X^2 \sum Y^2$$

Or $\left[\sum XY \right]^2 \leq \sum X^2 \sum Y^2$

Hence proved.



- correct proofs will always be accepted
- wrong proofs will be caught with “good” probability.



Part 4: Why Take This Course in the Age of LLMs?

➤ First, Let Us Admit the Power of LLMs

LLMs are excellent learning tools. They can:

- Answer questions immediately.
- Adapt an explanation to a student's background.
- Generate examples and exercises.
- Give feedback on code and attempted proofs.
- Translate between intuition and formal notation.
- Remain available at almost any time.

**This course should not pretend that LLMs do not exist.
We should learn to use them productively and critically.**

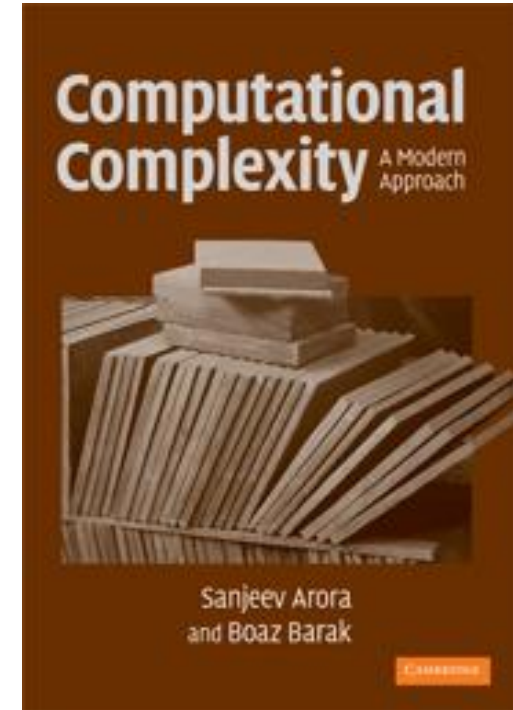
➤ Then Why Not Learn Everything from an LLM?

A complexity theory textbook could contain hundreds of pages.

A beginning student may not know:

- Which topics are appropriate for undergraduates.
- Which definitions are central and which are technical.
- Which proofs should be understood in full.
- Which sections may be skipped on a first reading.
- Which obsolete topics should be deleted.
- Which modern topics should be added.

Answers highly depend on an instructor's mathematical aesthetic.



Asking an LLM to “teach me complexity theory” does not itself determine a coherent curriculum.

➤ The Subtlety of Question Formalization

To obtain a useful answer from an LLM, you often need to ask a **sufficiently precise** question.

But a student's genuine state may be:

- "I understand every line, but I do not understand the proof."
- "I do not know exactly where I became confused."
- "The definition is clear, but why is it defined this way?"
- "I can follow this reduction, but I could never invent it."
- "Something feels wrong, but I cannot formulate an objection."

These are not yet well-formalized questions.

Before answering a question, a teacher may need to help the student discover what the real question is.

What a Human Instructor Can Observe

In a classroom, an instructor may notice:

- Many students making the facial expression of confusion.
- A long silence after a supposedly simple step.
- A question revealing an earlier misconception.
- Several incorrect solutions sharing one hidden assumption.
- Students using a definition correctly without understanding its purpose.

The instructor can then:

- Ask diagnostic questions.
- Change the example or draw a picture.
- Revisit an earlier concept.
- Adjust the remaining lectures.

A Course Provides an Environment

Knowing that high-quality information is available is not the same as learning it:

- A regular schedule.
- Carefully ordered material.
- Assignments and deadlines.
- Feedback on your own reasoning.
- A reason to continue when the topic becomes difficult.

Human Interaction is part of learning:

- Arguing about whether a proof is correct.
- Explaining an idea to a classmate.
- Learning which mistakes are common.
- Experiencing surprise together.
- Developing confidence in presenting and communicating technical ideas.

The goal is not “human instructor versus AI.”

The goal is to combine tools, structure, and human interaction to learn more effectively.



Part 5: What Is Bad About This Course?

➤➤ An Honest Warning: No Immediate Payoff

This course may not produce an immediately visible software/programming skill.

You may not add the following to your résumé immediately:

- a new mobile-development framework;
- a new web-development library;
- a new cloud deployment platform;
- a new programming language;
- a new user-interface tool.

The benefit is usually:

- long-term rather than immediate;
- conceptual rather than product-specific;
- reflected in your reasoning rather than one concrete artifact.

➤ The Benefit Can Be Difficult to Recognize

Foundations often improve performance indirectly.

Gradually, you may will notice that you have become better at:

- Identifying the essential part of a problem.
- Rejecting an unrealistic requirement.
- Separating worst-case and typical-case behavior.
- Distinguishing a theorem from empirical evidence.
- Connecting apparently unrelated tasks.
- Deciding when to seek an exact algorithm.

These improvements may become visible only after:

- Taking advanced courses.
- Entering graduate school to pursuit master or PhD
- Reading research papers.
- Encountering a task that resists ordinary optimization.

Let's Start!

No PowerPoint slides anymore
The instructor will use the whiteboard